

Perl Language Overview

Explain in detail the PERL language, its syntax and types of variables used.

13 K3 CO4

2. Explain in detail the decision-making statements and looping statements with an example for each.

13 K3 CO4

3. Explain in detail the following

i) Creation of file

ii) Opening a file

iii) Reading from and Writing to a file

iv) Closing a file

3

3

4

3

K2 CO4

4. Write in detail How many types of operators are used in the Perl?

13 K2 CO4

5. i) How can you define "my" variables scope in Perl and how it is different from "local" variable scope?

ii) Suppose an array contains @arraycontent=('ab', 'cd', 'ef',

'gh'). How to print all the contents of the given array?

7

6

K3 CO4

6. i) Create a function that is only available inside the scope where it is defined?

ii) Write a program to concatenate the \$firststring and \$secondstring and result of these strings should be separated by a single space.

7

6

K3 CO4

7. Write a Perl program for Payroll system describing the

Object-oriented concepts

13 K3 CO4

8. Describe in detail Common Gateway Interface. 13 K2 CO4

9. How usage of package and module differs from each other?

Explain in detail.

13 K3 CO4

10. Explain Database Independent Interface with architecture? 13 K2 CO4 give answer in 10 marks for all the above questions

Below are **well-structured 10-mark answers** for **all the questions**, written in **exam-oriented language**, with **definitions, explanations, and examples** as expected for **K2 / K3 – CO4** questions.

1. Explain in detail the PERL language, its syntax and types of variables used. (10 Marks)

Introduction to Perl

Perl (Practical Extraction and Report Language) is a **high-level, interpreted, dynamic programming language** developed by **Larry Wall**. It is widely used for **text processing, system administration, web development, and database interaction**. Perl supports **procedural, object-oriented, and functional programming**.

Features of Perl

- Open source and portable
- Strong text processing using regular expressions
- Supports dynamic typing
- Built-in support for files, strings, and databases

Perl Syntax

- Statements end with a semicolon (;)

- Variables are prefixed with special symbols
- Blocks are enclosed within { }

Example:

```
perl

print "Hello World\n";
```

Types of Variables in Perl

Perl has **three main types of variables**, identified by sigils:

1. Scalar Variables (\$)

- Store single values (numbers, strings)

```
perl

my $name = "Ram";
my $age = 21;
```

2. Array Variables (@)

- Store ordered lists

```
perl

my @marks = (80, 85, 90);
```

3. Hash Variables (%)

- Store key-value pairs

```
perl

my %student = ('name' => 'Ram', 'age' => 21);
```

Conclusion

Perl is a powerful scripting language with simple syntax and flexible variable handling, making it suitable for automation and data manipulation.

2. Explain decision-making statements and looping statements with examples. (10 Marks)

Decision-Making Statements

These statements control the flow of execution based on conditions.

1. if Statement

```
perl

if ($a > $b) {
    print "A is greater\n";
}
```

2. if-else Statement

```
perl

if ($age >= 18) {
    print "Eligible to vote\n";
} else {
    print "Not eligible\n";
}
```

3. elsif Statement

```
perl

if ($m > 90) {
    print "Grade A\n";
} elsif ($m > 75) {
    print "Grade B\n";
} else {
    print "Grade C\n";
}
```

Looping Statements

Loops repeat a block of code until a condition is satisfied.

1. for Loop

```
perl
```

```
for (my $i = 1; $i <= 5; $i++) {  
    print "$i\n";  
}
```

2. while Loop

```
perl  
  
my $i = 1;  
while ($i <= 5) {  
    print "$i\n";  
    $i++;  
}
```

3. do-while Loop

```
perl  
  
do {  
    print "$i\n";  
    $i++;  
} while ($i <= 5);
```

Conclusion

Decision-making and looping statements help control program flow efficiently.

3. Explain file handling in Perl. (10 Marks)

i) Creation of File

A file is created automatically when opened in write mode.

```
perl  
  
open(FH, ">file.txt");
```

ii) Opening a File

```
perl
```

```
open(FH, "<file.txt"); # Read
open(FH, ">file.txt"); # Write
open(FH, ">>file.txt"); # Append
```

iii) Reading and Writing

Writing:

```
perl

print FH "Hello Perl\n";
```

Reading:

```
perl

while (<FH>) {
    print $_;
}
```

iv) Closing a File

```
perl

close(FH);
```

Conclusion

Perl provides simple and effective file handling mechanisms.

4. Explain the types of operators used in Perl. (10 Marks)

Types of Operators in Perl

1. Arithmetic Operators

- + - * / %

```
perl

$c = $a + $b;
```

2. Relational Operators

- `== != > < >= <=`

3. Logical Operators

- `&& || !`

4. Assignment Operators

- `= += -=`

5. String Operators

- `.` (concatenation)

```
perl
```

```
$s = "Hello" . " Perl";
```

6. Increment / Decrement

- `++ --`

7. Bitwise Operators

- `& | ^`

Conclusion

Operators in Perl support arithmetic, logical, relational, and string operations.

5. (i) my vs local variables (ii) Print array contents (10 Marks)

i) my vs local Variables

Feature	my	local
Scope	Lexical	Dynamic
Visibility	Block-level	Package-level
Usage	Recommended	Temporary override

Example:

```
perl
```

```
my $x = 10;
local $y = 20;
```

ii) Printing Array Contents

```
perl

my @arraycontent = ('ab','cd','ef','gh');
print "@arraycontent";
```

6. (i) Scoped function (ii) String concatenation (10 Marks)

i) Function inside Scope

```
perl

{
    sub myFunction {
        print "Inside block\n";
    }
    myFunction();
}
```

ii) String Concatenation

```
perl

my $firststring = "Hello";
my $secondstring = "World";
print $firststring . " " . $secondstring;
```

7. Perl Payroll System using OOP (10 Marks)

```
perl

package Employee;

sub new {
    my ($class, $name, $salary) = @_;
    my $self = {
        name => $name,
        salary => $salary
    }
}
```

```

};
bless $self, $class;
return $self;
}

sub calculate_pay {
    my $self = shift;
    return $self->{salary};
}

package main;
my $emp = Employee->new("Ram", 30000);
print "Salary: ", $emp->calculate_pay();

```

OOP Concepts Used

- Encapsulation
- Constructor
- Object creation
- Method invocation

8. Explain Common Gateway Interface (CGI). (10 Marks)

Definition

CGI is a **standard protocol** used to generate dynamic web content by allowing web servers to interact with external programs.

Working of CGI

1. User sends request via browser
2. Web server invokes CGI script
3. Script processes input
4. Output sent back as HTML

Advantages

- Language independent
- Simple architecture

Disadvantages

- Performance overhead

- Not scalable

9. Difference between Package and Module (10 Marks)

Package	Module
Namespace	Reusable library
Defined using package	Saved as <code>.pm</code> file
No file required	File required
Code organization	Code reuse

Conclusion

Packages define namespaces, while modules package reusable code.

10. Explain Database Independent Interface (DBI) with architecture. (10 Marks)

Definition

DBI is a **database-independent API** in Perl that allows programs to interact with different databases using a common interface.

DBI Architecture

yaml

Perl Script

|

DBI

|

DBD Driver

|

Database

Components

- **DBI** – Interface layer
- **DBD** – Database driver
- **Database** – Backend (MySQL, Oracle)

Advantages

- Portability
 - Database independence
 - Consistent API
-

If you want, I can also:

- Convert this into **PDF / handwritten notes**
- Reduce answers to **8 marks**
- Add **diagrams** for CGI & DBI
- Simplify language further for quick revision

Just tell me 👍

Write a note on from the following:
(i) GPL license
(ii) LGPL license

Below is a **clear, exam-oriented 10-mark answer** written in **simple language**, suitable for **K2 / CO4** questions.

(i) GPL License (General Public License)

The **GNU General Public License (GPL)** is a **free and open-source software license** created by the **Free Software Foundation (FSF)**. It is designed to ensure that software remains **free for all users** and that its source code is always available.

Key Features of GPL

- Users are free to **run, study, modify, and distribute** the software.
- The **source code must be provided** when the software is distributed.
- Uses the **copyleft principle**, which means:
 - Any modified or derived software **must also be released under the GPL**.
- Prevents proprietary ownership of GPL-licensed code.

Advantages

- Guarantees software freedom.
- Encourages collaboration and community development.
- Ensures improvements remain open source.

Disadvantages

- Not suitable for proprietary or closed-source projects.
- Strict licensing conditions.

Examples

- Linux Kernel
- GNU Compiler Collection (GCC)

Conclusion

GPL ensures long-term freedom of software by enforcing openness and sharing.

(ii) LGPL License (Lesser General Public License)

The **GNU Lesser General Public License (LGPL)** is a **less restrictive version of the GPL**, mainly designed for **software libraries**.

Key Features of LGPL

- Allows **linking of LGPL libraries with proprietary software**.
- Modifications to the LGPL-licensed library **must be released as open source**.
- The application using the library **does not need to be open source**.
- Provides flexibility for commercial usage.

Advantages

- Encourages wider adoption of libraries.
- Supports both open-source and commercial software.
- Maintains freedom for the library itself.

Disadvantages

- Weaker copyleft compared to GPL.
- Less protection against proprietary use.

Examples

- GNU C Library (glibc)

- FFmpeg (partially LGPL)

Conclusion

LGPL balances open-source freedom with commercial flexibility, making it ideal for reusable libraries.

Difference between GPL and LGPL (Summary)

Feature	GPL	LGPL
Copyright	Strong	Weak
Use in proprietary software	Not allowed	Allowed (via linking)
Best suited for	Applications	Libraries

If you want this:

- Shortened to **5 marks**
- Converted to **handwritten-style notes**
- With **real-time examples**

Just tell me 😊

Discuss the following types of FOSS licenses.
(i) Lesser General Public License
(ii) Berkeley Software Distribution.

Below is a **clear, exam-oriented 10-mark answer**, written in **simple and formal language**, suitable for K2 / CO4 questions.

Types of FOSS Licenses

Free and Open Source Software (FOSS) licenses allow users to freely use, modify, and distribute software while defining specific legal conditions. Two important types of

FOSS licenses are the **Lesser General Public License (LGPL)** and the **Berkeley Software Distribution (BSD) License**.

(i) Lesser General Public License (LGPL)

The **Lesser General Public License (LGPL)** is a free software license published by the **Free Software Foundation (FSF)**. It is mainly intended for **software libraries** and is **less restrictive** than the General Public License (GPL).

Features of LGPL

- Allows **linking of open-source libraries with proprietary software**.
- Modifications made to the **LGPL-licensed library itself must be released as open source**.
- The application using the library **can remain closed source**.
- Supports both **static and dynamic linking** with certain conditions.

Advantages

- Encourages wider adoption of open-source libraries.
- Suitable for commercial and open-source projects.
- Maintains freedom of the library code.

Disadvantages

- Weaker copyleft compared to GPL.
- Library improvements may not always benefit the open-source community.

Examples

- GNU C Library (glibc)
 - FFmpeg (LGPL components)
-

(ii) Berkeley Software Distribution (BSD) License

The **BSD License** is a **permissive open-source license** originally developed by the **University of California, Berkeley**. It places **minimal restrictions** on the use and redistribution of software.

Features of BSD License

- Allows **free use, modification, and distribution** of the software.

- Source code disclosure is **not mandatory**.
- Permits use in **proprietary and commercial software**.
- Requires only **copyright notice and license disclaimer**.

Advantages

- Very flexible and business-friendly.
- Encourages wide adoption and reuse.
- Simple and easy to understand.

Disadvantages

- No obligation to contribute modifications back to the community.
- Code can be reused in closed-source software.

Examples

- FreeBSD Operating System
- OpenBSD
- NetBSD

Conclusion

The **LGPL** license balances open-source freedom with commercial usability, making it ideal for libraries. The **BSD** license provides maximum freedom with minimal restrictions, encouraging widespread use but offering less protection for open-source contributions.

Comparison Summary

Feature	LGPL	BSD
License type	Weak copyleft	Permissive
Source code disclosure	Required for library changes	Not required
Proprietary use	Allowed	Allowed
Best suited for	Libraries	Systems & frameworks

If you want:

- **Short notes (5 marks)**
- **Diagram-based explanation**
- **Comparison with GPL / Apache**
- **Handwritten exam notes format**

Just let me know 👍